

分块代码:

```
/*EHnotgod-----
.....#####.....#.....#.....
.....#.....#.....#.....
.....#####.....#####.....
.....#.....#.....#.....
.....#####.....#.....#.....
*/
#include <bits/stdc++.h>
using namespace std;
// debug 宏: 在编译时定义 -DLOCAL 会启用打印, 否则为 no-op
#ifdef LOCAL
template<class T> string ts(const T& v){
    stringstream ss; ss << v; return ss.str();
}
template<class A,class B> string ts(const pair<A,B>& p){ return "
("+ts(p.first)+","+ts(p.second)+")"; }
template<class T> string ts(const vector<T>& v){ string s="{"; for (auto &x:v) s
+= ts(x)+","; return s+"}"; }
template<class T, class C, class A> string ts(const set<T,C,A>& v){ string s="{";
for (auto &x:v) s += ts(x)+","; return s+"}"; }
template<class T, class C, class A> string ts(const multiset<T,C,A>& v){ string
s="{"; for (auto &x:v) s += ts(x)+","; return s+"}"; }
template<class K,class V,class C, class A> string ts(const map<K,V,C,A>& m){
string s="{"; for (auto &kv:m) s += ts(kv.first)+":"+ts(kv.second)+","; return
s+"}"; }

void dbg_out(){ cerr << "\n"; }
template<class H, class...T> void dbg_out(const H& h, const T&...t){
    cerr << " " << ts(h);
    dbg_out(t...);
}
#define debug(...) cerr<< "["<<#__VA_ARGS__<<"]:"<<,dbg_out(__VA_ARGS__)
#else
#define debug(...) 114514
#endif
#define endl "\n"
typedef long long ll;
#define range(i, a, b) for (int i = (a); i < (b); ++i)
#define def(name, ...) auto name = [&](__VA_ARGS__)
int n, m;

// vector<int> a(n, 0);
// vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
void solve() {
    vector<int> ru = {4, 7, 44, 47, 74, 77, 444, 447, 474, 477, 744, 747, 774,
777, 4444, 4447, 4474, 4477, 4744, 4747, 4774, 4777, 7444, 7447, 7474, 7477,
7744, 7747, 7774, 7777};
    vector<int> isyun(10010, 0);
    for (int u: ru) isyun[u] = 1;
    cin >> n >> m;
    vector<int> a(n);
    vector<vector<int>> freq((int)sqrt(n) + 10, vector<int>(10010, 0));
    vector<int> lazy((int)sqrt(n) + 10, 0);
```

```

vector<int> block(n + 1);
int da = (int)sqrt(n);
range(i, 0, n){
    cin >> a[i]; block[i] = i / da;
    freq[block[i]][a[i]] += 1;
}
// debug(a, se);
range(i, 0, m){
    string s; cin >> s;
    if (s == "count"){
        int ans = 0;
        int l, r; cin >> l >> r; l -= 1; r -= 1;
        int l_blk = block[l];
        int r_blk = block[r];
        if (l_blk == r_blk){
            range(j, l, r + 1){
                if (isyun[a[j] + lazy[l_blk]]){
                    ans += 1;
                }
            }
            cout << ans << endl;
        }
        else {
            range(j, l, l_blk * da + da){
                if (isyun[a[j] + lazy[l_blk]]){
                    ans += 1;
                }
            }
            range(j, r_blk * da, r + 1){
                if (isyun[a[j] + lazy[r_blk]]){
                    ans += 1;
                }
            }
            range(j, l_blk + 1, r_blk){
                for (int u: ru){
                    if (u - lazy[j] >= 0){
                        ans += freq[j][u - lazy[j]];
                    }
                }
            }
            cout << ans << endl;
        }
    }
    else {
        int l, r, d; cin >> l >> r >> d; l -= 1; r -= 1;
        int l_blk = block[l];
        int r_blk = block[r];
        if (l_blk == r_blk){
            range(j, l_blk * da, min(l_blk * da + da, n)){
                freq[l_blk][a[j]] -= 1;
                a[j] += lazy[l_blk];
                if (j <= r and j >= l) a[j] += d;
                freq[l_blk][a[j]] += 1;
            }
            lazy[l_blk] = 0;
        }
        else {
            range(j, l_blk * da, l_blk * da + da){

```



```

// template<class T> string ts(const vector<T>& v){ string s="{"; for (auto &x:v)
s += ts(x)+","; return s+"}"; }
// template<class T, class C, class A> string ts(const set<T,C,A>& v){ string s="
{"; for (auto &x:v) s += ts(x)+","; return s+"}"; }
// template<class T, class C, class A> string ts(const multiset<T,C,A>& v){
string s="{"; for (auto &x:v) s += ts(x)+","; return s+"}"; }
// template<class K,class V,class C, class A> string ts(const map<K,V,C,A>& m){
string s="{"; for (auto &kv:m) s += ts(kv.first)+":"+ts(kv.second)+","; return
s+"}"; }
// void dbg_out(){ cerr << "\n"; }
// template<class H, class...T> void dbg_out(const H& h, const T&...t){
//     cerr << " " << ts(h);
//     dbg_out(t...);
// }
// #define debug(...) cerr<<"["<<#__VA_ARGS__<<"]:"<<,dbg_out(__VA_ARGS__)

// 2.1 点
// 2.1.1基本定义
struct pt{
    T x, y;
    pt operator+(pt p){return{x+p.x, y+p.y};}
    pt operator-(pt p){return{x-p.x, y-p.y};}
    pt operator*(T d){return{x*d, y*d};}
    pt operator/(T d){return{x/d, y/d};} //注意不能除以0
    bool operator<(const pt& a) const {
        if (x != a.x) return x < a.x;
        return y < a.y;
    }
    bool operator==(const pt& other) const { return x == other.x && y == other.y; }
}
    bool operator!=(const pt& other) const { return !(*this == other); }
};
T sq(pt p) {return p.x*p.x+ p.y*p.y;}
T absv(pt p){return sqrt(sq(p));}
ostream&operator<<(ostream&os, pt p){
    return os <<"("<< p.x<<","<< p.y<<")";
}
int sgn(T x){return(T(0)< x)-(x <T(0));}
// 2.1.3基本变换: 平移
pt translate(pt v, pt p){
    return p + v;
}
// 2.1.4基本变换: 缩放
pt scale(pt c,T factor, pt p){
    return c +(p - c)* factor;
}
// 2.1.5基本变换: 旋转 (逆时针)
pt rot(pt p,T a){
    return{p.x*cos(a)- p.y*sin(a), p.x*sin(a)+ p.y*cos(a)};
}
pt perp(pt p){
    return{-p.y, p.x};
}
// 2.1.6 点积
T dot(pt v, pt w){
    return v.x* w.x+ v.y* w.y;
}
bool isPerp(pt v, pt w){
    return dot(v, w)==0;
}

```

```

}
// 2.1.7 叉积
T cross(pt v, pt w){return v.x* w.y- v.y* w.x;}
T angle(pt v, pt w){
    return abs(atan2(cross(v, w), dot(v, w)));
}
T orient(pt a, pt b, pt c){return cross(b - a, c - a);}
bool inAngle(pt a, pt b, pt c, pt p){
    assert(orient(a, b, c)!=0);
    if(orient(a, b, c)<0)swap(b, c);
    return orient(a, b, p)>=0&&orient(a, c, p)<=0;
}
T orientedAngle(pt a, pt b, pt c){
    if(orient(a, b, c)>=0){return angle(b - a, c - a);}
    else{return 2* M_PI -angle(b - a, c - a);}
}
}
// 2.1.8 极角排序
bool half(pt p){
    return p.y>0||(p.y==0&& p.x<0);
}
void polarSort(vector<pt>&v){
    sort(v.begin(), v.end(), [](pt v, pt w){
        return make_tuple(half(v),0,sq(v))<
            make_tuple(half(w),cross(v,w),sq(w));
    });
}

// 2.2 直线
// 2.2.1 基本定义
struct line{
    pt v; T c;
    line(pt v, T c):v(v),c(c){} // 点斜式
    line(T a, T b, T c):v({b,-a}),c(c){} // 标准式(ax+by+c=0)
    line(pt p, pt q):v(q-p),c(cross(v,p)){} // 两点式
};
// 2.2.2点与直线的位置关系
T side(line ln, pt p){
    return cross(ln.v, p)- ln.c;}
// 2.2.3点到直线的距离
T dist(line ln, pt p){return abs(side(ln, p))/absv(ln.v);}
// 2.2.4过一点的垂线
line perpThrough(line ln, pt p){return{p, p +perp(ln.v)}};
// 2.2.5沿直线排序
bool cmpProj(line ln, pt p, pt q){return dot(ln.v, p)<dot(ln.v, q);}
// 2.2.6沿指定方向平移
line translate(line ln, pt t){return{ln.v, ln.c +cross(ln.v, t)}};
// 2.2.7直线交点
bool inter(line l1, line l2, pt &out){
    T d =cross(l1.v, l2.v); if(d == 0)return false;
    out =(l2.v* l1.c- l1.v* l2.c)/ d;
    return true;
}
// 2.2.8正交投影与反射
pt proj(line ln, pt p){return p -perp(ln.v)*side(ln, p)/sq(ln.v);}
pt refl(line ln, pt p){return p -perp(ln.v)*2*side(ln, p)/sq(ln.v);}
// 2.2.9点到直线的平方距离
T sqDist(line ln, pt p) {

```

```

        return side(ln, p) * side(ln, p) / sq(ln.v);
    }
// 2.3 线段
// 2.3.0 定义?没有定义, 线段不就是两个点吗
// 2.3.1 点在线段为直径的圆上
bool inDisk(pt a, pt b, pt p){return dot(a - p, b - p)<=0;}
// 2.3.2 点在线段上
bool onSegment(pt a, pt b, pt p){
    return orient(a, b, p)==0 && inDisk(a, b, p);
}
// 2.3.3 线段与线段交点
bool properInter(pt a, pt b, pt c, pt d, pt &out){
    T oa =orient(c, d, a), ob =orient(c, d, b),
    oc =orient(a, b, c), od =orient(a, b, d);
    if(oa * ob <0&& oc * od <0){
        out =(a * ob - b * oa)/(ob - oa);
        return true;
    }
    return false;
}
set<pt>inters(pt a, pt b, pt c, pt d){
    pt out;
    if(properInter(a, b, c, d, out))return{out};
    set<pt> s;
    if(onSegment(c, d, a)) s.insert(a);
    if(onSegment(c, d, b)) s.insert(b);
    if(onSegment(a, b, c)) s.insert(c);
    if(onSegment(a, b, d)) s.insert(d);
    return s;
}
}
// 如果|s|=1则只有一个交点; 如果|s|=2则说明相交的区域是一条线段, 并且中存的是其他两个端点

// 2.3.4 点到线段的距离
T segPoint(pt a, pt b, pt p){
    if(a != b){
        line l(a, b);
        if(cmpProj(l, a, p)&& cmpProj(l, p, b))
            return dist(l, p);
        return min(absv(p - a),absv(p - b));
    }
    return absv(p - a);
}

// 2.3.5 线段到线段的距离
T segSeg(pt a, pt b, pt c, pt d){
    pt dummy;
    if(properInter(a, b, c, d, dummy))return 0;
    return min({segPoint(a, b, c),segPoint(a, b, d),
        segPoint(c, d, a),segPoint(c, d, b)});
}

// 2.4 多边形
// 2.4.1 多边形面积
T areaPolygon(vector<pt>p){
    T area =0.0;
    for(int i =0, n = p.size(); i < n; i++){
        area +=cross(p[i], p[(i +1)% n]);
    }
    return abs(area)/2.0;
}

```

```

}
// 2.4.2 射线法: 判断点是否在多边形内部
bool above(pt a, pt p){
    return p.y >= a.y;
}
bool crossesRay(pt a, pt p, pt q){
    return (above(a, q) - above(a, p)) *
        orient(a, p, q) > 0;
}
// 这个strict可以自己调控, 如果 strict=true (默认值), 点在边上返回 false。如果
// strict=false, 点在边上返回 true。
bool inPolygon(vector<pt> p, pt a, bool strict=true){
    int numCrossings = 0;
    for(int i = 0, n = p.size(); i < n; i++){
        if(onSegment(p[i], p[(i + 1) % n], a)) return !strict;
        numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
    }
    return numCrossings & 1;
}
// 2.4.3 凸包
bool turn_left(pt a, pt b, pt c) {
    return sgn(cross(b - a, c - a)) > 0;
}
vector<pt> convex_hull(vector<pt> a) {
    if (a.size() <= 2) return a;
    pt base = *min_element(a.begin(), a.end());
    sort(a.begin(), a.end(), [&](auto u, auto v) {
        return make_tuple(half(u - base), 0) < make_tuple(half(v - base), cross(u
- base, v - base));
    });
    vector<pt> ret;
    for (auto i : a) {
        while (ret.size() > 1 && !turn_left(ret[ret.size() - 2], ret[ret.size() -
1], i))
            ret.pop_back();
        ret.push_back(i);
    }
    return ret;
}

// 2.4.4 CCW判点-严格判断点 p 是否在凸多边形 A (CCW) 内部, 但是A必须逆时针排列
bool pointInConvexStrict(const vector<pt>& A, const pt& p) {
    int n = (int)A.size();
    if (n < 3) return false;
    if (sgn(orient(A[0], A[1], p)) <= 0) return false;
    if (sgn(orient(A[0], A[n-1], p)) >= 0) return false;

    int l = 1, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) >> 1;
        if (sgn(orient(A[0], A[mid], p)) > 0) l = mid;
        else r = mid;
    }
    return sgn(orient(A[l], A[l+1], p)) > 0;
}

// 2.5 圆
// 2.5.1 外接圆, 返回的是外心! 别问我为啥不返回半径, 要算的话, 自己逮个点算不就行了

```

```

pt circumCenter(pt a, pt b, pt c){
    b = b - a, c = c - a;
    assert(cross(b, c)!=0);
    return a +perp(b *sq(c)- c *sq(b))/cross(b, c)/2;
}

// 2.5.2圆与直线的交点
int circleLine(pt o,T r, line ln,pair<pt, pt>&out){
    T h2 = r * r - sqDist(ln, o);
    if(h2 >= 0){
        pt p = proj(ln, o);
        pt h = ln.v*sqrt(h2)/absv(ln.v);
        out = {p - h, p + h};
    }
    return 1+sgn(h2);
}

// 2.5.3圆与圆的交点
int circleCircle(pt o1,T r1, pt o2,T r2, pair<pt, pt>&out){
    pt d = o2 - o1;
    T d2 =sq(d);
    if(d2 == 0){
        assert(r1 != r2);
        return 0;
    }
    T pd =(d2 + r1 * r1 - r2 * r2)/2;
    T h2 = r1 * r1 - pd * pd / d2;
    if(h2 >=0){ pt p = o1 + d * pd / d2, h =perp(d)*sqrt(h2 / d2); out = {p - h,
p + h};}
    return 1 + sgn(h2);
}

// 快捷CV处
// vector<int> a(n, 0);
// vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
void solve() {
    int n; cin >> n;
    vector<pt> a(n);
    for (int i = 0; i < n; i++){
        cin >> a[i].x; cin >> a[i].y;
    }
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int t;
    cin >> t;
    while (t--){
        solve();
    }
    return 0;
}

```

拉格朗日：


```

// 拉格朗日插值法  $O(n^2)$ 
#include <iostream>
#include <cstring>
#include <algorithm>
using namespace std;

#define LL long long
const LL mod=998244353;
LL n,k,ans;
LL x[2005],y[2005];

LL ksm(LL a,LL b){
    LL s=1;
    while(b){
        if(b&1)s=s*a%mod;
        a=a*a%mod;
        b>>=1;
    }
    return s;
}

int main(){
    cin>>n>>k;
    for(int i=1;i<=n;i++)cin>>x[i]>>y[i];
    for(int i=1;i<=n;i++){
        LL a=y[i],b=1;
        for(int j=1;j<=n;j++){
            if(i==j) continue;
            a=a*(k-x[j])%mod;
            b=b*(x[i]-x[j])%mod;
        }
        ans=(ans+a*ksm(b,mod-2)%mod)%mod;
    }
    cout<<(ans+mod)%mod;
}

```